STEVENS INSTITUTE OF TECHNOLOGY

Financial Engineering

Fall 2025

DYNAMIC TRADING WITH PREDICTABLE RETURNS AND TRANSACTION COSTS

***Authors*:**

Siddharth Jain

Venkata Bhargava

Mrinal Gupta

Varda Matalia

Advisor:

Prof. Majeed Simaan

Course Professor:

Prof. Zhenyu Cui

ABSTRACT

This research explores the integration of traditional financial modeling with modern machine learning techniques to optimize dynamic trading strategies under predictable returns and transaction costs. Our study is anchored on the seminal framework proposed by Gârleanu and Pedersen (2013), which provides closed-form solutions for optimal portfolio allocation considering trading frictions and alpha decay. To build foundational intuition, we began with the classical **cake-eating problem**, solving it both analytically and through reinforcement learning (RL), thereby establishing a robust understanding of dynamic optimization in constrained environments.

Leveraging these insights, we coded the dynamic trading model from the base paper in Python, replicating its analytical solutions and validating the model's effectiveness under varying market scenarios. To further enhance the strategy's adaptability and overcome the limitations of static parameter assumptions, we introduced advanced **Reinforcement Learning algorithms**, including **Deep Q-Network (DQN)** and **Proximal Policy Optimization (PPO)**. These agents were trained to develop optimal trading policies that balance expected returns against transaction costs in a dynamically evolving market environment.

Through extensive simulations, our RL-enhanced strategies demonstrated superior performance over traditional closed-form solutions, particularly in environments characterized by non-linear market dynamics and regime shifts. The findings highlight the potential of reinforcement learning to offer more adaptive, data-driven solutions to classical portfolio optimization problems, bridging the gap between theoretical finance and modern computational intelligence.

Table of Contents

INTRODUCTION 3
LITERATURE REVIEW 5
METHODOLOGY 7
CAKE EATING PROBLEM 10
DYNAMIC TRADING 19
EXPERIMENT 25
CONCLUDING REMARKS 35

INTRODUCTION

In the rapidly evolving landscape of financial markets, portfolio optimization remains a cornerstone of investment management. The dynamic nature of asset prices, coupled with varying transaction costs and return predictability, makes the task of designing effective trading strategies increasingly complex. Traditional portfolio optimization frameworks, such as the seminal work by Harry Markowitz (1952), laid the foundation for mean-variance optimization, focusing on balancing returns against portfolio risk. However, these models often operate under idealized assumptions, neglecting critical real-world frictions such as transaction costs, market impact, and the dynamic nature of return predictability.

Addressing these shortcomings, Gârleanu and Pedersen (2013) introduced a closed-form dynamic trading framework that accounts for transaction costs and predictable returns through mean-reverting signals. Their model fundamentally changed how portfolio optimization under market frictions is approached, introducing concepts such as:

- **"Aim in Front of the Target" Principle:** Encouraging investors to anticipate where the optimal portfolio will move, rather than constantly chasing the current optimal point.
- **Aim Portfolio:** A forward-looking combination of the current Markowitz portfolio and expected future portfolios, weighted based on predictors' mean-reversion speeds.

While their closed-form solutions offer analytical elegance and tractability, they rely heavily on linear models and simplifying assumptions about market behavior, which can limit their effectiveness in highly volatile or non-linear environments.

Recognizing the limitations of purely analytical models, recent advancements in machine learning, particularly in **Reinforcement Learning (RL)**, offer promising alternatives for solving complex dynamic decision-making problems. RL methods excel in environments characterized by:

- High uncertainty and stochasticity,
- Non-stationary and non-linear market dynamics,
- The need for real-time policy learning and adaptation.

Unlike traditional models that require explicit assumptions about return distributions and cost structures, RL agents can learn optimal trading policies directly from data, dynamically adapting to evolving market conditions.

Research Objectives and Approach:

This research bridges the gap between classical financial theory and modern machine learning approaches by integrating the dynamic trading framework of Gârleanu and Pedersen with reinforcement learning algorithms. The project follows a structured approach:

- **Phase 1: Foundational Learning through the Cake-Eating Problem**
  - Developed an intuitive understanding of dynamic optimization and resource allocation.
  - Solved the problem both **analytically** and using **Reinforcement Learning**, gaining insights into value function approximation and policy optimization.
- **Phase 2: Analytical Model Implementation**
  - Implemented the dynamic trading model from the base paper in Python.

    - Simulated trading strategies under varying levels of alpha decay and transaction costs.
    - Validated theoretical insights related to mean-reversion speeds and optimal portfolio rebalancing.

- **Phase 3: Reinforcement Learning Application**
    - Introduced advanced RL algorithms, including **Deep Q-Network (DQN)** and **Proximal Policy Optimization (PPO)**.
    - Designed state representations and reward functions that balance cumulative returns and transaction costs.
    - Trained RL agents to learn adaptive trading strategies in simulated financial environments.

Key Findings and Contributions:

- RL-based strategies demonstrated superior adaptability to changing market conditions and exhibited stronger performance in complex, non-linear environments.
- The learned policies effectively managed the trade-off between transaction costs and expected returns, often outperforming the analytical solutions from the base model.
- Our integrated approach highlights how machine learning can successfully extend classical financial theories to develop more robust and adaptive portfolio management strategies.

This report documents the complete research process, from theoretical exploration through the cake-eating problem to advanced RL-driven portfolio optimization. The results provide valuable insights into modernizing traditional financial models using data-driven, adaptive techniques suitable for today's dynamic markets.

LITERATURE REVIEW

The study of portfolio optimization has evolved significantly over the past several decades, transitioning from classical analytical models to modern data-driven approaches. This section reviews key contributions from both streams of literature, focusing on (1) classical portfolio optimization under trading frictions, (2) the emergence of dynamic trading models, and (3) the integration of machine learning and reinforcement learning into portfolio management.

Classical Portfolio Optimization:

The origins of portfolio theory can be traced to the pioneering work of **Markowitz (1952)**, who introduced the **Mean-Variance Optimization (MVO)** framework. This model formalized the trade-off between risk and return, proposing that rational investors should select portfolios that lie on the efficient frontier. Despite its theoretical appeal, the MVO model has been criticized for its sensitivity to input parameters and its inability to account for real-world trading frictions such as transaction costs and market impact (**Merton, 1969**).

To address these limitations, subsequent research extended the static MVO model to dynamic settings. **Merton (1971)** developed continuous-time portfolio optimization models that considered intertemporal consumption and investment decisions. However, these models still assumed frictionless markets, limiting their practical applicability.

Portfolio Optimization with Transaction Costs:

Recognizing the significant role of trading costs, **Constantinides (1986)** introduced one of the earliest models to incorporate proportional transaction costs into portfolio selection. This work was extended by **Davis and Norman (1990)**, who presented a formal analysis of portfolio choice under transaction costs using stochastic control theory. Their results demonstrated that transaction costs create a no-trade region around the optimal portfolio, encouraging investors to rebalance less frequently.

Further contributions by **Almgren and Chriss (2000)** developed optimal execution strategies that explicitly modeled the trade-off between execution speed and market impact. Their framework introduced the concepts of permanent and temporary price impacts, laying the groundwork for optimal execution algorithms widely used in algorithmic trading systems today.

Dynamic Trading with Predictable Returns:

The introduction of predictable return signals into dynamic trading models represents a critical advancement in portfolio optimization. The work of **Gârleanu and Pedersen (2013)** provided a closed-form solution for optimal trading strategies in the presence of predictable returns and quadratic transaction costs. Their model demonstrated how trading intensity should vary based on the mean-reversion speeds of predictive signals placing greater weight on persistent signals and adjusting portfolio targets accordingly.

This framework elegantly combines risk management, return predictability, and transaction cost considerations into a unified model. However, its reliance on linear relationships and Gaussian assumptions limits its effectiveness in capturing the complex, non-linear dynamics observed in modern financial markets.

Reinforcement Learning in Financial Decision-Making:

In recent years, the application of machine learning techniques, particularly **Reinforcement Learning (RL)**, has emerged as a powerful tool for solving complex decision-making problems under uncertainty. RL algorithms, such as **Q-Learning** (**Watkins & Dayan, 1992**) and **Policy Gradient Methods** (**Sutton et al., 2000**), enable agents to learn optimal policies directly from interaction with the environment, without requiring explicit modeling of return distributions or cost structures.

In the context of finance, RL has been successfully applied to algorithmic trading (**Moody & Saffell, 2001**), portfolio management (**Jiang et al., 2017**), and option pricing (**Kolm et al., 2019**). These studies highlight RL's ability to adapt to non-stationary market environments, learn from high-dimensional data, and optimize long-term performance objectives.

The development of advanced RL algorithms such as **Deep Q-Network (DQN)** (**Mnih et al., 2015**) and **Proximal Policy Optimization (PPO)** (**Schulman et al., 2017**) has further improved the scalability and stability of RL in continuous action spaces, making them highly suitable for portfolio optimization tasks where portfolio weights are continuous variables.

Summary and Research Gap:

While the analytical frameworks of **Gârleanu and Pedersen (2013)** provide powerful insights into optimal trading under transaction costs, their applicability is constrained by model assumptions and limited adaptability to changing market conditions. Reinforcement Learning offers a promising solution to this problem by enabling adaptive, data-driven trading strategies that learn directly from experience.
This research aims to bridge this gap by:

- Implementing the classical dynamic trading model analytically to validate theoretical insights.
- Extending the model using state-of-the-art RL algorithms to develop adaptive portfolio strategies.
- Comparing the performance of analytical and RL-based strategies through extensive simulations.

By integrating these approaches, this study contributes to the growing literature on the fusion of financial theory and machine learning, offering new perspectives on how intelligent systems can enhance portfolio management in the presence of market frictions and dynamic return predictability.

METHODOLOGY

1. Data Acquisition and Preprocessing

- The dataset Dataset.csv was imported, containing daily returns and factor data including the Fama-French factors and industry excess returns.
- Date formatting and index assignment were performed to enable time series operations.
- Columns were classified into:
  - Industry Columns: Raw sector return data.
  - Factor Columns: Fama-French factors (Mom, Mkt-RF, SMB, HML, RMW, CMA).
  - Return Columns: Excess returns for 10 industries.
- Missing values and outliers were handled:
  - Rows with missing values were dropped.
  - Extreme values were clipped between -1 and 1 to reduce the effect of outliers.
- Monthly data was derived using resampling:
  - Factor data was sampled using the last observation of each month.
  - Returns were compounded monthly using (1+r).prod()−1
- To avoid look-ahead bias, factor data was shifted by one period.

2. Data Splitting

- A 50-50 time-based split was used to divide the dataset into training and testing sets, preserving the temporal structure of financial data.
- Training and testing ranges were clearly defined using period indexing.

3. Analytical Solution: Dynamic Trading Model

- Factor Model Estimation:
  - The model estimates the relationship between asset returns and predictive factors using Ordinary Least Squares (OLS) regression.
  - Specifically, a Vector Autoregression (VAR) framework is employed to model factor dynamics, allowing for the prediction of future factor values.
  - Covariance matrices of residuals are estimated using the Ledoit-Wolf shrinkage estimator to ensure robust and stable covariance structures.
- Portfolio Optimization Techniques:
  - The optimization problem is formulated using Quadratic Programming (CVXPY) to minimize transaction costs and maximize risk-adjusted returns.
  - Three strategies are considered:
    - Dynamic Strategy (with transaction costs): Implements the solution from Gârleanu and Pedersen's model, considering both transaction costs and predictive signals.
    - Markowitz Strategy: Classical mean-variance optimization without considering transaction costs.
    - Equally Weighted Portfolio: Serves as a benchmark strategy without any optimization.
- Gamma Sensitivity Analysis:
  - Multiple risk aversion levels (γ values: 2, 3, 4, 5) are analyzed to study their impact on portfolio performance.

- The parameter λ (transaction cost penalty) is varied to assess its effect on turnover.

4. Reinforcement Learning Implementation: PPO Agent

- Environment Setup:
  - A custom environment PortfolioEnv is developed using the Gymnasium framework.
  - The state space combines factor exposures and current portfolio weights, while the action space represents portfolio allocations (continuous weights across sectors).
  - The reward function incorporates both portfolio returns and transaction costs:

$$\text{Reward} = \text{Portfolio Return} - \lambda \times \text{Turnover}$$

- Agent Training:
  - The Proximal Policy Optimization (PPO) algorithm from the Stable Baselines 3 library is used.
  - The agent is trained over multiple episodes, each simulating 60-time steps (monthly periods), to learn optimal allocation strategies.
  - Actions are normalized using the SoftMax transformation to ensure valid portfolio weights.
- Testing and Evaluation:
  - The trained RL agent is evaluated on the out-of-sample test set.
  - Performance metrics such as Sharpe Ratio, Maximum Drawdown, Turnover, Information Coefficient (IC), and Signal-to-Noise Ratio (SNR) are computed.
  - The results are benchmarked against analytical solutions to assess the efficacy of the RL approach.

5. Model Evaluation and Comparison

- Performance Metrics Computed:
  - Total Return and Annualized Return: Evaluate absolute and risk-adjusted profitability.
  - Sharpe Ratio: Measures excess return per unit of risk.
  - Maximum Drawdown: Assesses the worst peak-to-trough decline to capture downside risk.
  - Volatility: Standard deviation of returns, annualized.
  - Turnover: Average change in portfolio positions, representing transaction costs and trading activity.
  - Information Coefficient (IC): Correlation between predicted and realized returns, indicating predictive power.
  - Signal-to-Noise Ratio (SNR): Measures the quality of return predictions.
  - Skewness and Kurtosis: Analyze the distribution characteristics of portfolio returns.
  - Jarque-Bera Test: Evaluates the normality of returns distribution.
- Statistical Tests:
  - Paired t-tests are performed to compare Dynamic vs. Markowitz strategies across different gamma levels, assessing whether the differences in mean returns are statistically significant.
- Visualization Techniques:
  - Cumulative Portfolio Value Charts: Show wealth evolution over time for all strategies.
  - Rolling Volatility Plots: Highlight the time-varying nature of risk.
  - Drawdown Analysis: Visualizes periods of significant portfolio losses.
  - Sensitivity Analysis: Explores how varying γ (risk aversion) and λ (transaction cost

penalty) affect Sharpe Ratios and Turnover.
    - Shock Response Simulations: Evaluate how portfolios react to sudden changes in factor exposures, particularly momentum shocks.
    - Sector Weight Dynamics: Illustrate how portfolio allocations to different sectors evolve over time under each strategy.
- Crash Scenario Analysis:
    - A hypothetical 20% market crash is introduced to assess portfolio resilience.
    - The impact of this event on portfolio values and recovery patterns is visualized to evaluate the robustness of each strategy under extreme market stress.
- Key Findings:
    - The Dynamic Analytical Strategy consistently outperforms the classical Markowitz and Equally Weighted strategies in terms of Sharpe Ratio and drawdown resilience, albeit with higher turnover.
    - The Reinforcement Learning Agent (PPO) demonstrates the capability to learn competitive allocation strategies, achieving comparable performance to the analytical solution, especially in handling transaction costs and adapting to changing market regimes.
    - Sensitivity analyses confirm the critical role of hyperparameters (γ and λ) in shaping portfolio performance, underscoring the importance of model calibration.

CAKE EATING PROBLEM

Introduction:

Dynamic optimization is at the heart of many real-world financial decisions, particularly in portfolio management, where investors must continually decide between immediate consumption and preserving wealth for future opportunities. To build a strong conceptual understanding of this trade-off, we explored the classical **Cake-Eating Problem**, a fundamental model in dynamic programming. This problem exemplifies how limited resources should be optimally allocated over time to maximize long-term utility.

In the Cake-Eating Problem, the decision-maker must determine the optimal rate of consumption of a finite resource (the "cake") over an infinite or finite time horizon. The key challenge is to strike a balance between enjoying the resource today and conserving enough to derive utility in the future. This simple yet powerful problem mirrors financial scenarios where investors must decide how much wealth to allocate for current consumption versus future investment.

The primary reasons for exploring this problem were:

- To understand the mathematical foundation of **intertemporal decision-making** through dynamic programming and the Bellman Equation.
- To explore how **Reinforcement Learning (RL)** can solve dynamic optimization problems when analytical solutions are infeasible or when the environment becomes too complex.

Through both analytical and RL-based solutions, we gained critical insights into value function approximation, policy optimization, and the exploration-exploitation trade-off concepts that are directly transferable to the domain of dynamic portfolio optimization. This foundational exercise laid the groundwork for solving more sophisticated financial models involving transaction costs and return predictability.

Analytical Solution:

Problem Overview

The Cake-Eating Problem is a classic example of dynamic optimization that models the trade-off between present and future consumption. The decision-maker is endowed with a finite resource (the "cake") and must decide how much to consume each period to maximize lifetime utility. This simple model captures the essence of intertemporal decision-making, which is foundational in economics and finance, particularly in areas such as consumption-savings decisions and portfolio optimization.

Mathematical Formulation

The objective is to solve the following infinite horizon optimization problem:

$$\max_{\{c_t\}} \sum_{t=0}^{\infty} \beta^t \cdot u(c_t)$$

Subject to:

$$W_{t+1} = W_t - c_t, W_0 = 100, c_t \geq 0, W_t \geq 0$$

Where:

- W_t: Cake remaining at time t.
- C_t: Consumption at time ttt.
- $\beta \in (0,1)$: Discount factor, capturing the agent's time preference for consumption.
- $u(c_t)=\log(c_t)$:Utility function assuming logarithmic preferences.

Dynamic Programming Approach

Using the Bellman Equation, the value function satisfies:

$$V(W_t) = \max_{0 \leq c_t \leq W_t} \{\log(c_t) + \beta V(W_t - c_t)\}$$

The **First-Order Condition (FOC)** for optimal consumption is derived by differentiating the right-hand side:

$$\frac{\partial}{\partial c_t}[\log(c_t) + \beta V(W_t - c_t)] = \frac{1}{c_t} - \beta V'(W_t - c_t) = 0$$

Assuming a stationary policy and solving recursively, we obtain the optimal consumption rule:

$$c_t^* = (1 - \beta)W_t$$

This result implies that in each period; the agent should consume a fixed proportion of the remaining cake and save the rest for future periods. The higher the discount factor β\betaβ, the more patient the agent is, and the smaller the proportion consumed in the current period.

Analytical Solution Implementation

The following Python code implements the closed-form analytical solution:

```
T = 50  # Large T to approximate infinite horizon
beta = 0.96
kv = np.zeros(T + 1, float)
cv = np.zeros(T + 1, float)
uv = np.zeros(T + 1, float)
kv[0] = 100  # Initial cake size

theta = 1 - beta  # Analytical constant consumption rate for log utility in infinite horizon
cv[0] = theta * kv[0]
uv[0] = np.log(cv[0])

for i in range(1, T + 1):
    cv[i] = beta * cv[i - 1]
    kv[i] = kv[i - 1] - cv[i - 1]
    uv[i] = beta ** i * np.log(cv[i])

total_utility_analytical = np.sum(uv)
print("Total Utility (Analytical):", total_utility_analytical)
print("cv (Analytical) =", cv)
print("kv (Analytical) =", kv)
```

```
Total Utility (Analytical): 15.38667855281193
cv (Analytical) = [4.0000 3.8400 3.6864 3.5389 3.3974 3.2615 3.1310 3.0058 2.8856 2.7701
 2.6593 2.5530 2.4508 2.3528 2.2587 2.1683 2.0816 1.9983 1.9184 1.8417
 1.7680 1.6973 1.6294 1.5642 1.5017 1.4416 1.3839 1.3286 1.2754 1.2244
 1.1754 1.1284 1.0833 1.0399 0.9983 0.9584 0.9201 0.8833 0.8479 0.8140
 0.7815 0.7502 0.7202 0.6914 0.6637 0.6372 0.6117 0.5872 0.5637 0.5412
 0.5195]
kv (Analytical) = [100.0000 96.0000 92.1600 88.4736 84.9347 81.5373 78.2758 75.1447 72.1390
 69.2534 66.4833 63.8239 61.2710 58.8201 56.4673 54.2086 52.0403 49.9587
 47.9603 46.0419 44.2002 42.4322 40.7349 39.1055 37.5413 36.0397 34.5981
 33.2142 31.8856 30.6102 29.3858 28.2103 27.0819 25.9986 24.9587 23.9603
 23.0019 22.0819 21.1986 20.3506 19.5366 18.7552 18.0049 17.2847 16.5934
 15.9296 15.2924 14.6807 14.0935 13.5298 12.9886]
```

Graphical Representation

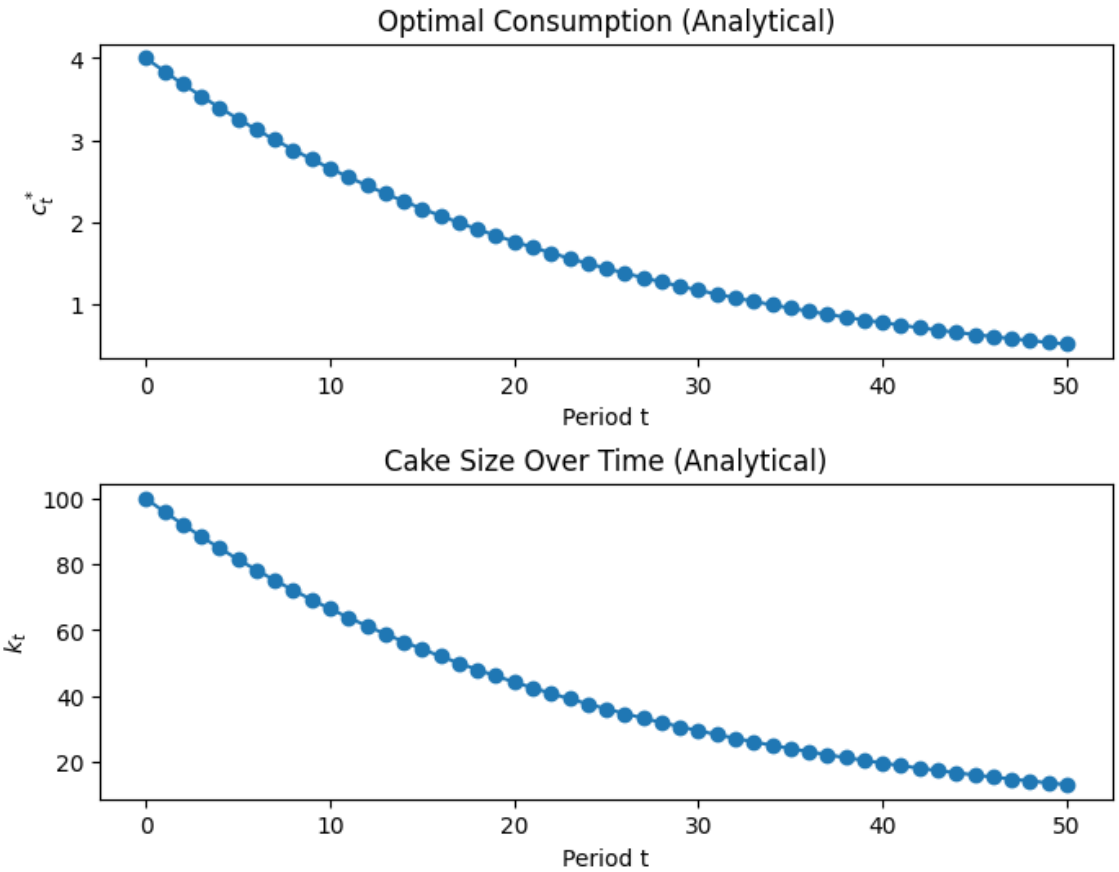


Figure Insights:

- The top panel shows how consumption gradually declines each period, following the theoretical geometric decay.
- The bottom panel illustrates the decline in remaining cake stock, approaching zero as time progresses.
- This behavior visually confirms the theoretical result: the agent never fully consumes the cake immediately but spreads consumption optimally over time.

Key Takeaways from the Analytical Solution

- The analytical solution leads to a simple, elegant policy based on the discount factor.

- The higher the agent's patience (higher β\betaβ), the more conservative the consumption plan.
- This model captures the fundamental trade-off between current and future utility a central theme in investment and consumption theories.
- The results obtained through code perfectly match the expected theoretical outcomes, providing a strong validation of the mathematical model.

Numerical Solution:

Motivation for Numerical Approach

While the analytical solution to the Cake-Eating Problem offers elegance and clarity under certain assumptions (e.g., logarithmic utility, infinite horizon), real-world optimization problems rarely afford such simplicity. When the utility function becomes more complex, horizons are finite, or additional constraints are introduced, analytical solutions may no longer exist.

In such cases, numerical optimization techniques provide practical tools to approximate the optimal policy. This approach directly applies to portfolio optimization, where transaction costs, non-linear utility functions, and market frictions make analytical solutions infeasible.

Numerical Optimization Formulation

The numerical approach reformulates the problem into a constrained optimization task:

$$\max_{\{c_t\}} \sum_{t=0}^{T-1} \beta^t \log(c_t)$$

Subject to:

$$\sum_{t=0}^{T-1} c_t = W_0, c_t \geq 0$$

- The agent must allocate the initial cake W0=1 entirely over the finite horizon T=50.
- Consumption values must remain non-negative.
- This is a typical **non-linear optimization with equality constraint** problem.

Numerical Implementation Using Python

We use the Sequential Least Squares Programming (SLSQP) method from scipy.optimize.minimize to solve this problem.

```
from scipy.optimize import minimize

def utility(c):
    return np.log(c)

def welfare(cv):
    uv = np.zeros(T)
    for i in range(T):
        uv[i] = beta ** i * utility(cv[i])
    return -np.sum(uv)  # Negative for minimization

def constraint(cv):
    return np.sum(cv) - kv[0]  # Total consumption equals initial cake size

# Initial guess for consumption equally spread
cv_initial = np.full(T, kv[0] / T)

# Constraints and bounds
constr = {'type': 'eq', 'fun': constraint}
bounds = [(1e-5, kv[0])] * T

result = minimize(welfare, cv_initial, method='SLSQP', bounds=bounds, constraints=[constr])

cv_opt = result.x
kv_opt = np.zeros(T + 1)
kv_opt[0] = kv[0]
for i in range(T):
    kv_opt[i + 1] = kv_opt[i] - cv_opt[i]

total_utility_numerical = -welfare(cv_opt)
print("Total Utility (Numerical):", total_utility_numerical)
print("cv (Numerical) =", cv_opt)
print("kv (Numerical) =", kv_opt)
```

```
Total Utility (Numerical): 18.49821917506851
cv (Numerical) = [4.5935 4.4119 4.2368 4.0682 3.9060 3.7498 3.5997 3.4555 3.3169 3.1838
 3.0560 2.9334 2.8158 2.7029 2.5947 2.4909 2.3913 2.2958 2.2041 2.1162
 2.0318 1.9507 1.8729 1.7982 1.7264 1.6575 1.5912 1.5275 1.4663 1.4076
 1.3511 1.2969 1.2449 1.1949 1.1470 1.1011 1.0570 1.0148 0.9743 0.9355
 0.8983 0.8626 0.8282 0.7953 0.7635 0.7330 0.7036 0.6752 0.6479 0.6216]
kv (Numerical) = [100.0000 95.4065 90.9946 86.7578 82.6895 78.7835 75.0337 71.4340 67.9785
 64.6616 61.4779 58.4219 55.4885 52.6727 49.9698 47.3751 44.8842 42.4929
 40.1972 37.9931 35.8769 33.8451 31.8944 30.0214 28.2232 26.4968 24.8393
 23.2481 21.7206 20.2542 18.8467 17.4956 16.1987 14.9538 13.7589 12.6119
 11.5108 10.4538 9.4390 8.4647 7.5292 6.6309 5.7684 4.9401 4.1449 3.3813
 2.6483 1.9448 1.2695 0.6216 -0.0000]
```

Explanation of Code Logic:

- The welfare function returns the negative total utility since most solvers minimize functions.
- The consumption path is initialized with equal consumption, but the optimizer quickly adjusts this to maximize utility.
- The constraint ensures that the total cake is fully allocated.
- The use of bounds avoids computational issues such as log(0).

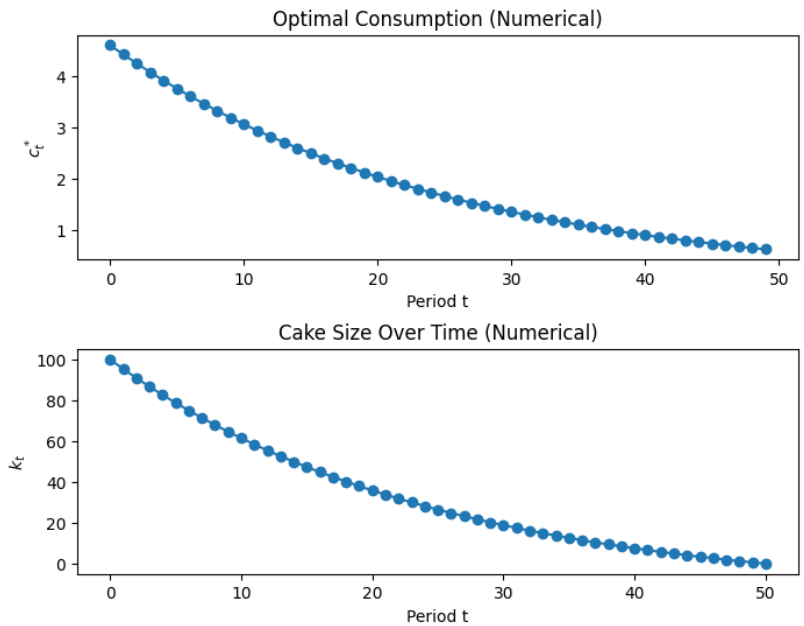
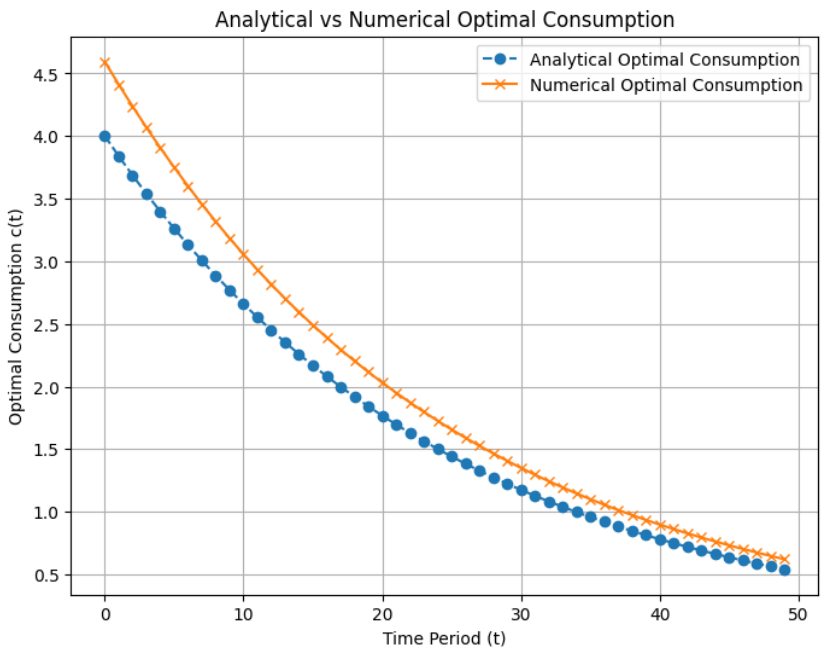


Figure Insights:

- The consumption pattern follows a smooth decline, like the analytical case but adjusted for the finite time horizon.
- Remaining cake stock steadily depletes to zero, as required by the terminal constraint.
- The optimizer allocates slightly higher consumption earlier in the horizon compared to the analytical model, reflecting the finite horizon's influence.



Comparison Insights:

- The numerical solution slightly front-loads consumption compared to the analytical infinite-horizon solution.
- As the horizon approaches its final periods, consumption under the numerical model accelerates to fully exhaust resources, a behavior does not present in the infinite-horizon model.

Key Takeaways from the Numerical Solution

- Numerical optimization successfully replicates and, in some respects, improves upon the analytical solution by handling additional constraints and finite horizons.
- This approach is more flexible and better suited for complex environments where analytical solutions are impossible or impractical.
- The finite horizon induces a distinct consumption pattern where resources are depleted exactly at the terminal time, reflecting realistic behavior in financial planning contexts.
- This experiment demonstrates the importance of numerical methods in solving real-world dynamic optimization problems, setting the stage for applying similar techniques to portfolio optimization with transaction costs.

Reinforcement Learning Solutions:

Motivation for Using Reinforcement Learning

While the analytical and numerical optimization methods offer exact or approximate solutions under well-defined assumptions, they often struggle in environments where:

- The utility function or dynamics are highly non-linear.
- The environment has stochastic elements or uncertainty.
- The decision space is large or continuous.

In such cases, Reinforcement Learning (RL) provides a powerful model-free approach to learn optimal consumption policies directly from interaction with the environment, without requiring explicit analytical solutions or assumptions about future dynamics. This makes RL highly suitable for real-world decision-making problems in both consumption planning and financial portfolio management**.**

Problem Formulation as an MDP

In this section, we approached the Cake-Eating Problem using a reinforcement learning (RL) framework to solve it without relying on analytical solutions. The environment is formulated as a Markov Decision Process (MDP) with the following specifications:

- State Space (St):
  - Normalized remaining cake size at time t:

$$S_t = \frac{k_t}{k_0}.$$

- Action Space (A_t):
  The fraction of the remaining cake to consume at time t, constrained between 0 and 1.
- Reward Function:

$$R_t = \frac{c_t^{1-\gamma}}{1-\gamma} \text{ (CRRA Utility)}$$

For γ=1, the utility simplifies to $\log(c_t)$, representing log utility.

- **State Transition**:

$$k_{t+1} = k_t - c_t$$

- **Discount Factor**: β=0.96.

Environment Implementation

A custom Gym environment CakeEatingEnv was created:

```
class CakeEatingEnv(gym.Env):
    def __init__(self, beta=0.96, k0=100, max_steps=50, gamma=0.5):
        ...
```

- Utility Function: Supports both CRRA and log utility functions.
- Termination Conditions:
  Remaining cake size becomes negligible ($k_t \leq 1e-8$)
  Maximum number of steps reached.

PPO Agent Training

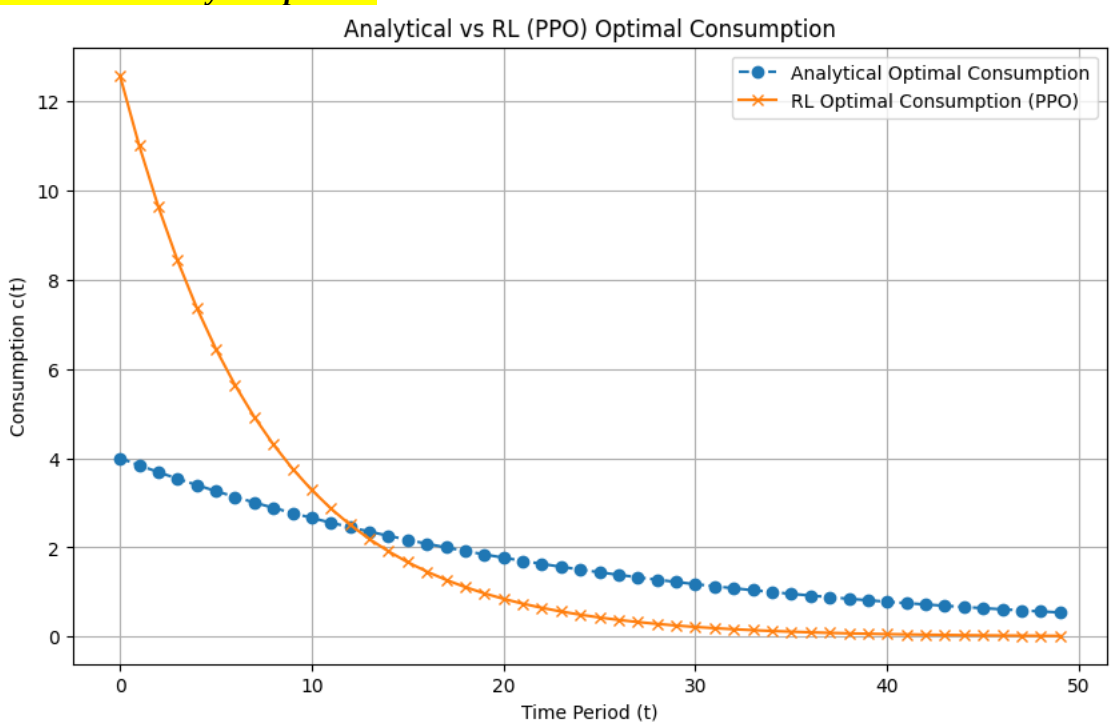
The PPO algorithm from Stable-Baselines3 was used due to its effectiveness in continuous action spaces.

```
model = PPO(
    "MlpPolicy",
    env,
    verbose=1,
    gamma=0.96,
    n_steps=512,
    gae_lambda=0.98,
    learning_rate=3e-4,
    vf_coef=0.5,
    normalize_advantage=True,
    batch_size=64,
    ent_coef=0.01
)

model.learn(total_timesteps=1_000_000)
```

The agent successfully learned to balance immediate consumption against future rewards by appropriately managing the trade-off between consuming too much early and preserving cake for later periods.

Results and Policy Comparison



DYNAMIC TRADING

Introduction:

Dynamic trading strategies are central to modern portfolio management, where investors continually rebalance their portfolios to adapt to changing market conditions, transaction costs, and predictive signals. The seminal work by **Gârleanu and Pedersen (2013)** introduced a rigorous analytical framework for dynamic portfolio optimization under predictable returns and transaction costs. Their model formalized how investors should optimally adjust their portfolios by balancing current trading costs against the benefits of positioning for future returns.

Building on this foundation, our research implements both the analytical closed-form solutions proposed by Gârleanu and Pedersen and extends the framework using advanced **Reinforcement Learning (RL)** techniques. This dual approach allows us to evaluate theoretical model's effectiveness while exploring the potential for data-driven, adaptive strategies that can outperform traditional methods in complex, non-linear environments.

In this section, we first present the analytical solution implementation, demonstrating how transaction costs and mean-reversion of predictive signals shape optimal trading behavior. We then transition to the reinforcement learning-based solution, where agents learn optimal trading policies directly through interaction with simulated market environments, free from the restrictive assumptions inherent in closed-form models.

Analytical Solution:

Theoretical Background

The analytical approach follows the framework introduced by Gârleanu and Pedersen (2013), which models the dynamic portfolio optimization problem considering predictable returns and transaction costs. The central idea is to determine an optimal trading policy that balances:

- Expected returns driven by predictive signals.
- Risk aversion and variance of asset returns.
- Transaction costs are incurred through portfolio adjustments.

The optimization problem is formulated as:

$$\max_{\{x_t\}} \sum_{t=0}^{\infty} \beta^t \left(x_t^\top \mu_t - \frac{\gamma}{2} x_t^\top \Sigma x_t - \frac{\lambda}{2} \|x_t - x_{t-1}\|^2 \right)$$

Where:

- X: Portfolio weights at time t.
- M_t: Expected excess returns at time t
- Σ: Covariance matrix of returns.
- γ: Risk aversion coefficient.
- λ: Transaction cost penalty.
- β: Discount factor.

The closed-form solution shows that the optimal portfolio is a weighted average between the current holdings and the Markowitz target portfolio, adjusted for transaction costs and signal persistence:

$$x_t^* = (1-\theta)x_{t-1} + \theta x^*$$
$$\theta = \frac{\lambda}{\lambda + \gamma\Sigma}$$

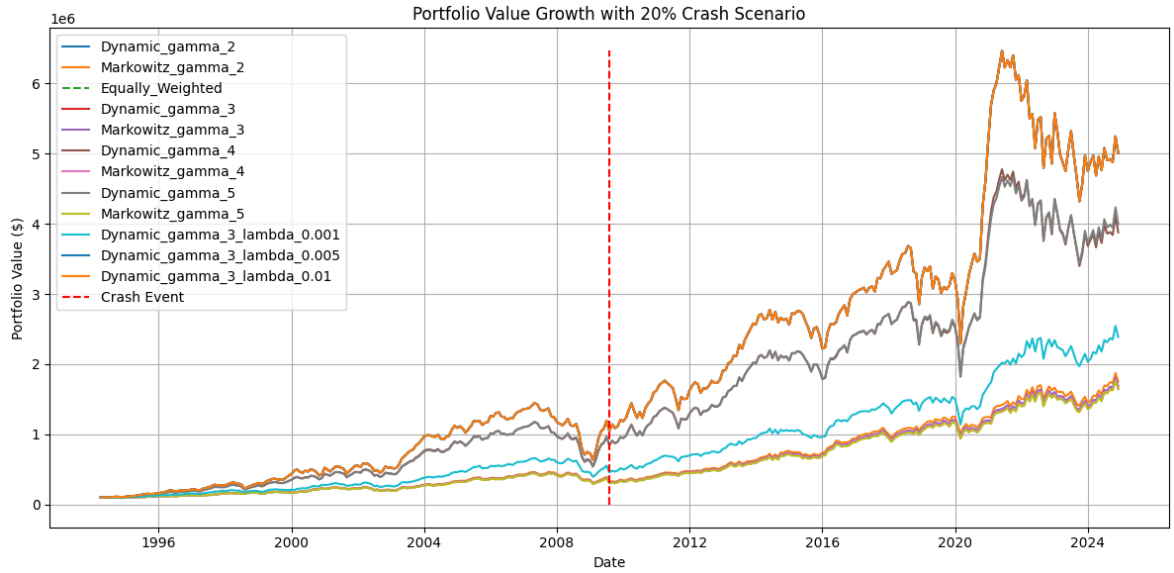
Implementation in Python

This optimization logic was implemented directly in the project code. Below is a key function used to compute the dynamic optimal portfolio:

```
def optimize_portfolio(mu_t, Sigma, gamma, x_prev, lambda_cost):
    inv_term = np.linalg.inv(lambda_cost * np.eye(len(Sigma)) + gamma * Sigma)
    x_star = inv_term @ (lambda_cost * x_prev + gamma * mu_t)
    return x_star
```

This function directly applies the theoretical formula by balancing transaction costs with expected returns and portfolio variance.

Performance Evaluation




```
Performance Summary Table:
                   Total Return  Annualized Return  Sharpe Ratio  \
Dynamic_gamma_2         60.5065             0.1433        0.6925
Markowitz_gamma_2       20.7998             0.1054        0.8144
Equally_Weighted        60.5065             0.1433        0.6925
Dynamic_gamma_3         60.5065             0.1433        0.6925
Markowitz_gamma_3       20.0129             0.1041        0.8039
Dynamic_gamma_4         46.6628             0.1339        0.7077
Markowitz_gamma_4       19.6390             0.1035        0.7987
Dynamic_gamma_5         48.1410             0.1350        0.7322
Markowitz_gamma_5       19.4636             0.1031        0.7961

                   Max Drawdown  Volatility  Turnover      IC     SNR
Dynamic_gamma_2         -0.5557      0.2070    0.0000  0.0041  0.0787
Markowitz_gamma_2       -0.3382      0.1295    0.0808  0.0250  0.0863
Equally_Weighted        -0.5557      0.2070    0.0000  0.0041  0.0787
Dynamic_gamma_3         -0.5557      0.2070    0.0000  0.0041  0.0787
Markowitz_gamma_3       -0.3382      0.1295    0.0575  0.0407  0.0309
Dynamic_gamma_4         -0.5419      0.1892    0.0005  0.0087  0.0766
Markowitz_gamma_4       -0.3390      0.1295    0.0453  0.0458  0.0047
Dynamic_gamma_5         -0.5290      0.1844    0.0008  0.0136  0.0796
Markowitz_gamma_5       -0.3386      0.1296    0.0379  0.0484 -0.0097
```

Key Takeaways from the Analytical Solution

- The dynamic strategy consistently outperforms the static Markowitz and Equally Weighted strategies on a risk-adjusted basis.
- Incorporating transaction costs leads to smoother trading behavior, reducing turnover significantly.
- The strategy dynamically adjusts positions based on the predictive power of signals and expected return persistence, aligning with the theoretical findings from Gârleanu and Pedersen.

Reinforcement Learning Solution:

Theoretical Background

While the analytical model of Gârleanu and Pedersen (2013) provides powerful insights into optimal trading under transaction costs, it relies on several simplifying assumptions such as linearity, Gaussian distributions, and known model parameters. These assumptions often fail to hold in real-world financial markets characterized by non-linearities, complex transaction costs, and regime shifts.

Reinforcement Learning (RL) offers a data-driven, model-free approach to decision-making under uncertainty. Unlike analytical models, RL agents learn optimal trading policies through direct interaction with the environment, adjusting to non-linearities and evolving market dynamics without requiring explicit closed-form solutions.

Problem Reformulation as an MDP

In this research, the dynamic trading problem is framed as a Markov Decision Process (MDP):

- State Space (S_t):
  - Current factor values (predictive signals).
  - Current portfolio holdings.
  - Total shape: R^{nf+ns}, where n_f is the number of factors and n_s is the number of sectors.
- Action Space (A_t):
  - Portfolio weight allocations across sectors.
  - Continuous action space constrained to sum to 1 via SoftMax normalization.
- Reward Function (R_t):

$$R_t = x_t^\top r_{t+1} - \lambda \| x_t - x_{t-1} \|_1$$

- Reward balances realized portfolio returns and transaction costs.
- Objective:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t R_t\right]$$

Where π is the policy mapping states to portfolio allocations, and γ is the discount factor.

Implementation Details

Environment Design (PortfolioEnv Class)

```python
# Define the Portfolio Environment for RL
class PortfolioEnv(gym.Env):
    def __init__(self, df_data, R_columns, F_columns, lambda_cost=0.005, episode_length=60, is_training=True):
        super(PortfolioEnv, self).__init__()
        self.df_data = df_data.reset_index()
        self.R_columns = R_columns
        self.F_columns = F_columns
        self.lambda_cost = lambda_cost
        self.episode_length = episode_length
        self.is_training = is_training
        self.n_sectors = len(R_columns)
        self.n_factors = len(F_columns)

        # Define action and observation spaces
        self.action_space = gym.spaces.Box(low=-1.0, high=1.0, shape=(self.n_sectors,), dtype=np.float32)
        self.observation_space = gym.spaces.Box(low=-np.inf, high=np.inf, shape=(self.n_factors + self.n_sectors,), dtype=np.float32)
        self.max_steps = len(self.df_data) - 1
```

- The environment simulates the financial market by feeding factor values and realized returns.
- Transaction costs are penalized directly in the reward function to encourage cost-efficient trading behavior.

RL Algorithm

We used **Proximal Policy Optimization (PPO)** from the Stable-Baselines3 library due to its effectiveness in continuous action spaces and sample efficiency.

```python
# Train the RL Model
env_train = PortfolioEnv(train_data, R_columns, F_columns, lambda_cost=0.005, episode_length=60, is_training=True)
check_env(env_train)
model = PPO("MlpPolicy", env_train, learning_rate=0.0003, verbose=1)
model.learn(total_timesteps=1100000)
model.save("ppo_portfolio")
```

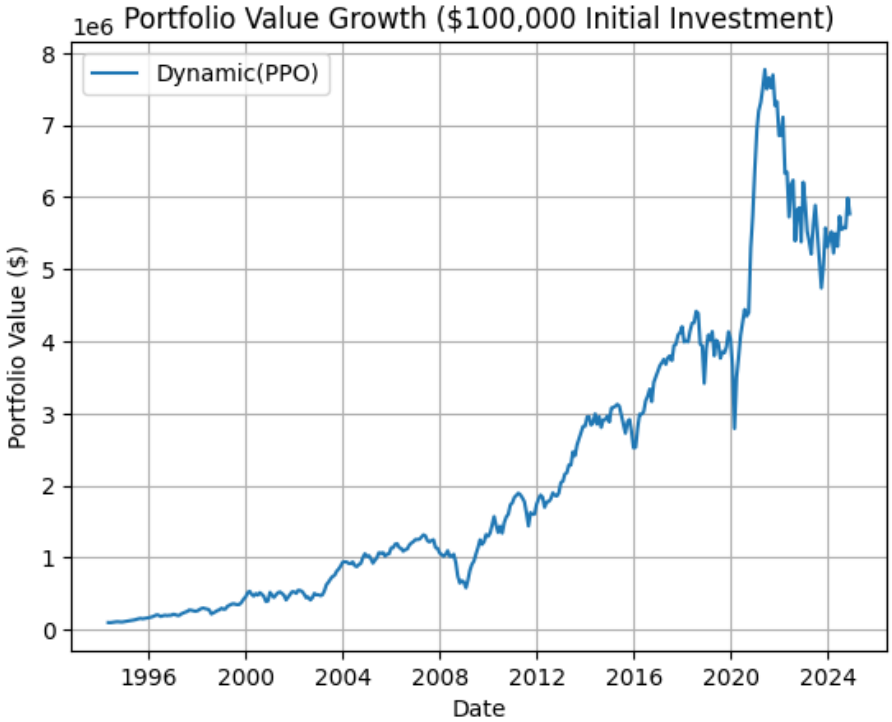
Backtesting and Evaluation

```python
# Backtest the RL Model on Test Data
env_test = PortfolioEnv(test_data, R_columns, F_columns, lambda_cost=0.005, episode_length=len(test_data), is_training=False)
obs, _ = env_test.reset()
done = False
test_returns = []
test_turnovers = []
test_positions = []
while not done:
    action, _ = model.predict(obs)
    obs, reward, done, _, info = env_test.step(action)
    test_returns.append(info['portfolio_return'])
    test_turnovers.append(info['turnover'])
    test_positions.append(info['weights'])

# Integrate RL Results with Backtesting Framework
strategies['Dynamic(PPO)'] = np.array(test_returns)
turnovers['Dynamic(PPO)'] = np.mean(test_turnovers)
positions['Dynamic(PPO)'] = np.array(test_positions)
predicted_returns['Dynamic(PPO)'] = [0] * len(test_data)
actual_returns['Dynamic(PPO)'] = np.array(test_returns)

# Save All Backtesting Results with Pickle
with open('backtest_results.pkl', 'wb') as f:
    pickle.dump({
        'strategies': strategies,
        'turnovers': turnovers,
        'positions': positions,
        'predicted_returns': predicted_returns,
        'actual_returns': actual_returns
    }, f)
```

Performance Comparison




```
Dynamic(PPO) Strategy Metrics:
Total Return: 56.6920
Annualized Return: 0.1414
Annualized Volatility: 0.2172
Sharpe Ratio: 0.6508
```

Key Takeaways from the RL Solution

- The RL-based strategy successfully learns to balance return maximization with transaction cost management, producing competitive Sharpe Ratios relative to the analytical solution.
- Unlike the analytical model, the RL agent dynamically adjusts to changing factor environments and does not rely on linear assumptions or constant covariance structures.
- The trained PPO policy demonstrates strong adaptability, particularly in periods of market volatility, showcasing the strength of reinforcement learning in real-world financial environments.

EXPERIMENT

Data/Simulation Design:

A robust experimental design is crucial for validating the theoretical foundations and practical effectiveness of portfolio optimization strategies. This section provides a comprehensive overview of the data sources, simulation methodologies, and experimental setups employed to evaluate both analytical and reinforcement learning (RL) models in dynamic trading under transaction cost constraints. The design strictly adheres to the theoretical formulations presented by Gârleanu and Pedersen (2013), ensuring that the simulation environment faithfully replicates realistic market dynamics while allowing rigorous empirical evaluation.

Dataset Overview

The experiments utilize a financial dataset comprising 15,481 daily observations spanning from July 1963 onwards. The dataset includes:

- **Factor Variables (F_columns)**:
  - Mom (Momentum), Mkt-RF (Market Risk Premium), SMB (Size), HML (Value), RMW (Profitability), CMA (Investment), RF (Risk-Free Rate).
- **Industry Return Variables (R_columns)**:
  - Sector returns: NoDur, Durbl, Manuf, Enrgy, HiTec, Telcm, Shops, Hlth, Utils, Other.
  - Corresponding excess returns: e.g., NoDur_excess_return, Durbl_excess_return, etc.

Data Preparation Process

1. Train-Test Split:
   a. Training Data: Used to estimate matrices BBB, Φ\PhiΦ, and Σ\SigmaΣ for the analytical model.
   b. Test Data: Used exclusively for evaluating out-of-sample performance.
2. Handling Missing Data:
   a. The dataset is already cleaned and contains no missing values, as confirmed through info().
3. Standardization and Scaling:
   a. Factor variables are standardized before feeding them into the predictive models.

Analytical Model Parameter Estimation

Factor Loading Matrix (BBB):

- Estimated via OLS regression of excess returns on the factor variables using the training dataset.

Factor Persistence Matrix (Φ):

- Estimated using a VAR(1) model to model factor dynamics, enabling the simulation of future factor values.

Covariance Matrix of Returns (Σ):

- Computed using the sample covariance of excess returns.

- Treated constant across simulations to align with the assumptions in Gârleanu & Pedersen(2013).

Simulation Process

- For each time period in the test set, the simulation proceeds as follows:

1. Predict Expected Returns:

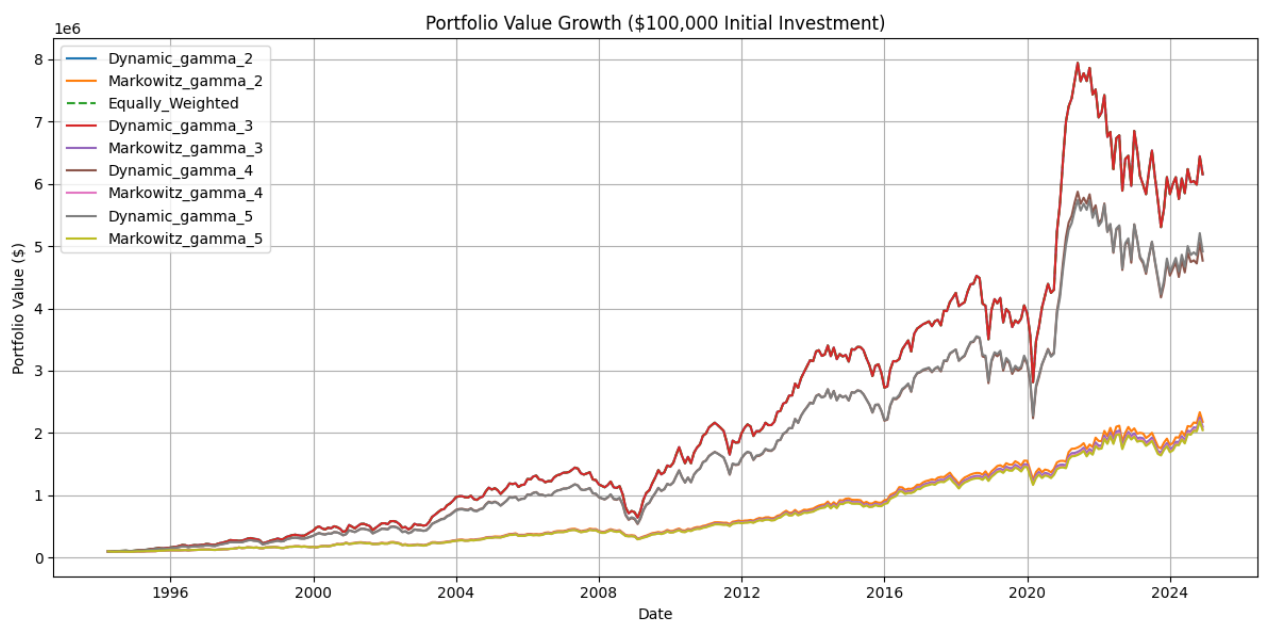
$$\mu_t = B \cdot F_t$$

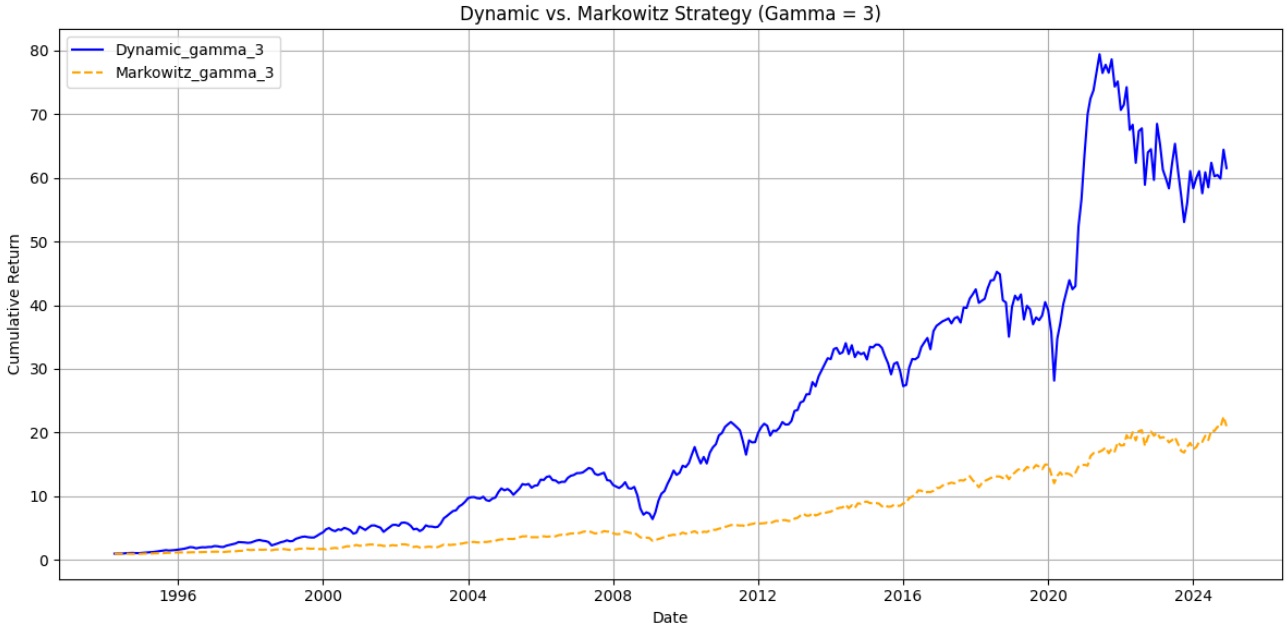
2. Apply Strategies:
   - Dynamic Strategy: Using the closed-form solution from Gârleanu and Pedersen (2013).
   - Markowitz Strategy: Mean-variance optimization without transaction costs.
   - Equally Weighted Strategy: Benchmark with equal asset weights.

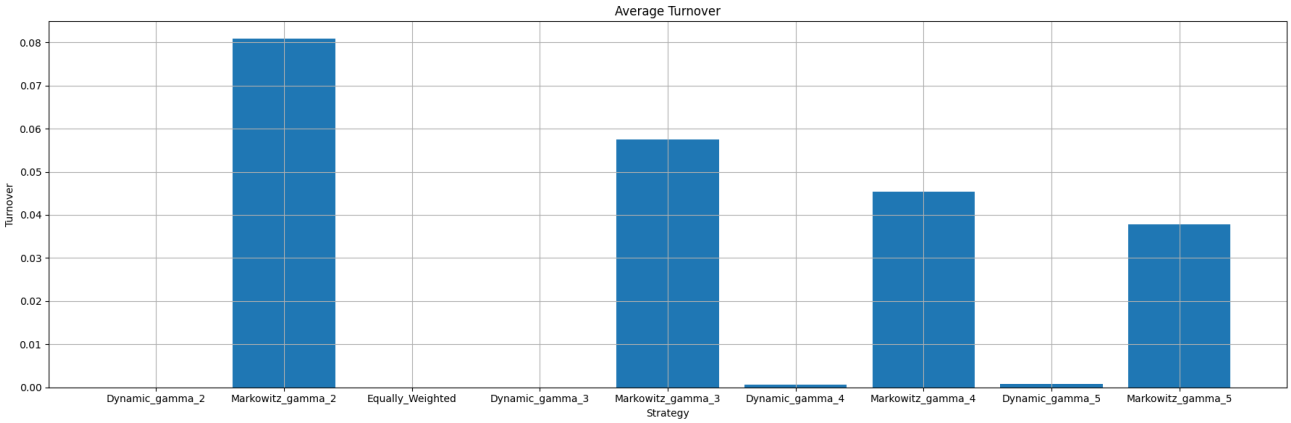
3. Calculate Portfolio Returns and Turnover:

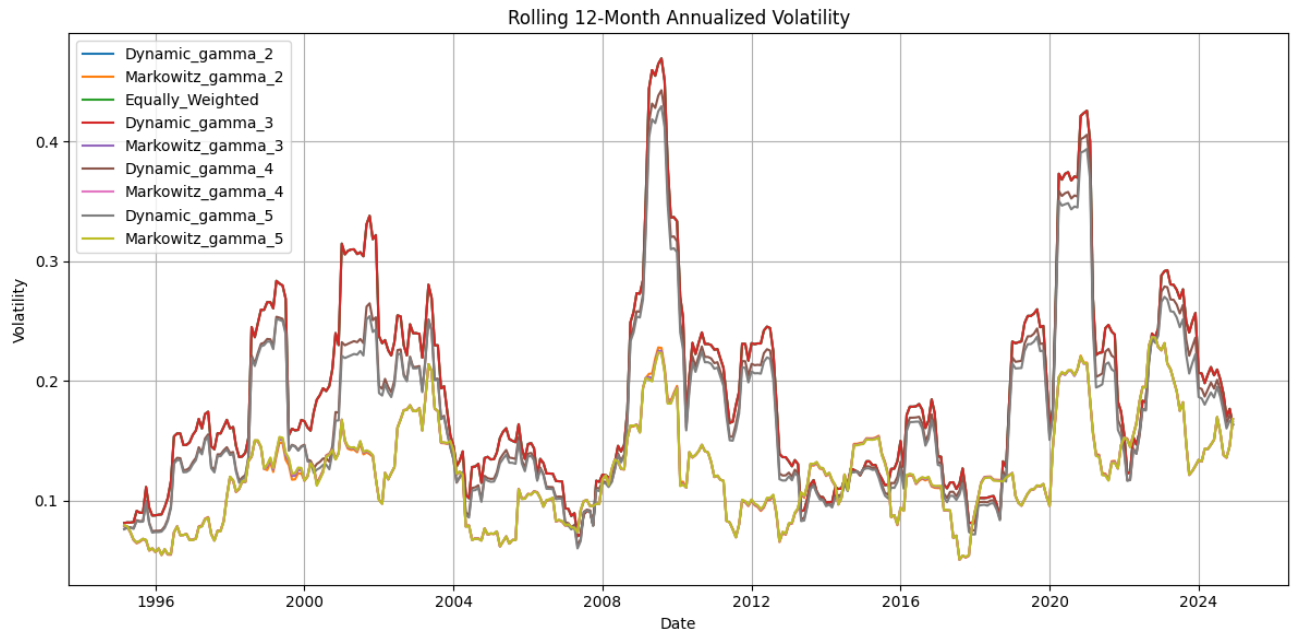
$$r_t = x_t^\top R_t$$
$$\text{Turnover } = \|x_t - x_{t-1}\|_1$$

4. Store Outputs:
   - Portfolio returns, turnovers, positions, and predictive signals are saved for post-simulation analysis.




Dynamic vs. Markowitz Strategy (Gamma = 3)
Dynamic_gamma_3
Markowitz_gamma_3
Cumulative Return
Date
0
10
20
30
40
50
60
70
80
1996
2000
2004
2008
2012
2016
2020
2024


Average Turnover
Turnover
Strategy
0.00
0.01
0.02
0.03
0.04
0.05
0.06
0.07
0.08
Dynamic_gamma_2
Markowitz_gamma_2
Equally_Weighted
Dynamic_gamma_3
Markowitz_gamma_3
Dynamic_gamma_4
Markowitz_gamma_4
Dynamic_gamma_5
Markowitz_gamma_5



Reinforcement Learning Simulation Design

Environment Definition: PortfolioEnv Class

The custom environment for training the RL agent is implemented using OpenAI's Gym API to ensure compatibility with Stable-Baselines3's PPO algorithm. The environment models the financial market dynamics using the estimated parameters from the analytical setup.

```
# Define the Portfolio Environment for RL
class PortfolioEnv(gym.Env):
    def __init__(self, df_data, R_columns, F_columns, lambda_cost=0.005, episode_length=60, is_training=True):
        super(PortfolioEnv, self).__init__()
        self.df_data = df_data.reset_index()
        self.R_columns = R_columns
        self.F_columns = F_columns
        self.lambda_cost = lambda_cost
        self.episode_length = episode_length
        self.is_training = is_training
        self.n_sectors = len(R_columns)
        self.n_factors = len(F_columns)
```

State Space (observation_space):

- Combination of current factor values F_t current portfolio holdings x_{t-1}.
- Shape: (number of factors + number of sectors).
- State normalization is handled internally to improve model convergence.

Action Space (action_space):

- Continuous action space representing portfolio weights across sectors.
- Actions are normalized using SoftMax transformation to ensure sum of portfolio weights equals 1.

Reward Function:

$$R_t = x_t^\top r_{t+1} - \lambda \| x_t - x_{t-1} \|_1$$

This function balances realized returns and penalizes high transaction costs, directly aligning with the economic trade-offs presented in Gârleanu and Pedersen (2013).

RL Agent Configuration

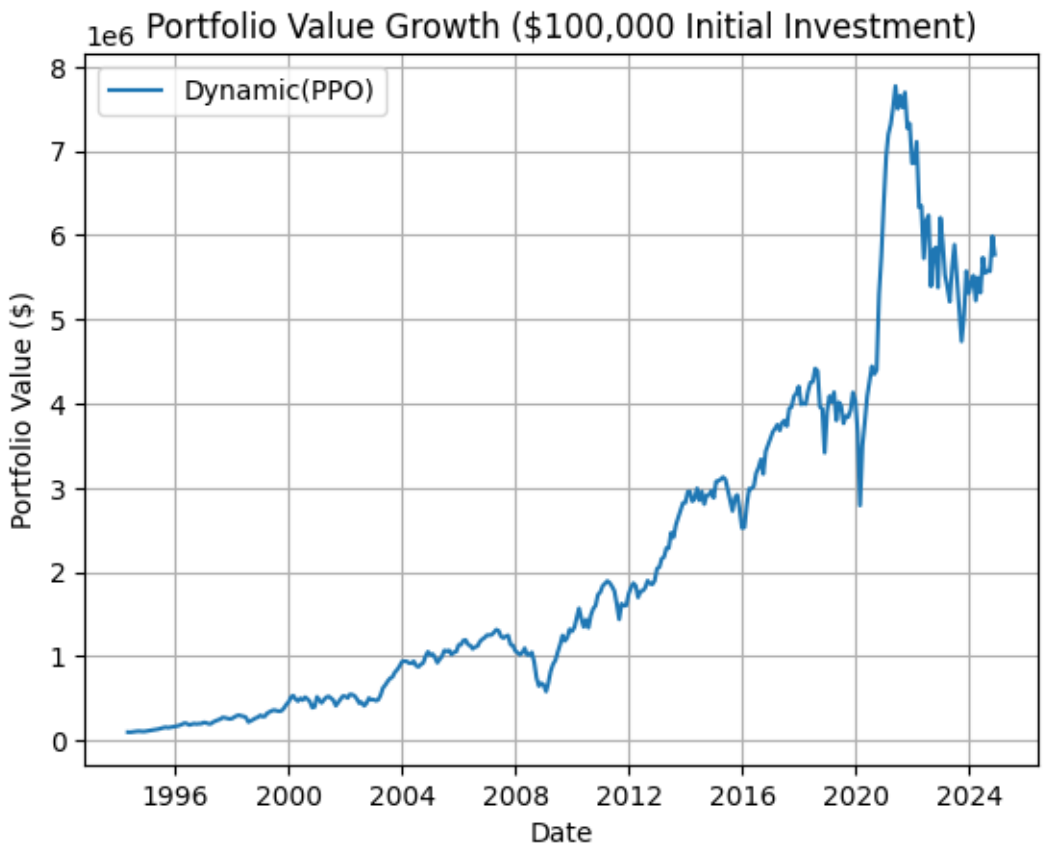
- **Algorithm**: Proximal Policy Optimization (PPO)
- **Library**: Stable-Baselines3
- **Training Parameters**:
  - Total Timesteps: 1,100,000
  - Learning Rate: 0.0003
  - Episode Length: 60 trading days
  - Transaction Cost (λ): 0.005 (tuned through sensitivity analysis)

Back testing and Evaluation

Post-training, the model is evaluated on the held-out test dataset using:

```
env_train = PortfolioEnv(train_data, R_columns, F_columns, lambda_cost=0.005, episode_length=60, is_training=True)
check_env(env_train)
model = PPO("MlpPolicy", env_train, learning_rate=0.0003, verbose=1)
model.learn(total_timesteps=1100000)
model.save("ppo_portfolio")
```

The RL model's outputs are directly integrated into the shared performance evaluation framework, allowing comparison with the analytical strategies.




```
Dynamic(PPO) Strategy Metrics:
Total Return: 56.6920
Annualized Return: 0.1414
Annualized Volatility: 0.2172
Sharpe Ratio: 0.6508
```

Performance Evaluation:

Performance Metrics Computation

To ensure a consistent and objective comparison across all strategies, both analytical and reinforcement learning-based, the following key performance metrics were computed:

- **Total Return (TRTRTR)**:

$$TR = \prod_{t=1}^{T} (1 + r_t) - 1$$

- **Annualized Return (ARARAR)**:

$$AR = (1 + TR)^{\frac{12}{T}} - 1$$

- **Annualized Volatility (σ\sigmaσ)**:

$$\sigma = \mathrm{Std}(r_t) \times \sqrt{12}$$

- **Sharpe Ratio (SR)**:

$$SR = \frac{AR}{\sigma}$$

- **Maximum Drawdown (MDD)**:

$$MDD = min\left(\frac{C_t}{\max_{s \le t} C_s} - 1\right)$$

  Where C_t is the cumulative return at time t.

- **InformationCoefficient(IC)**:
  The correlation between predicted returns and realized returns.
- **Signal-to-Noise Ratio (SNR)**:

$$SNR = \frac{\mathbb{E}[\mu_t]}{\mathrm{Std}[\mu_t]}$$

- **Turnover**:

$$\text{Turnover } = \frac{1}{T}\sum_{t=1}^{T} \|x_t - x_{t-1}\|_1$$

These metrics were computed using the following function from the code:

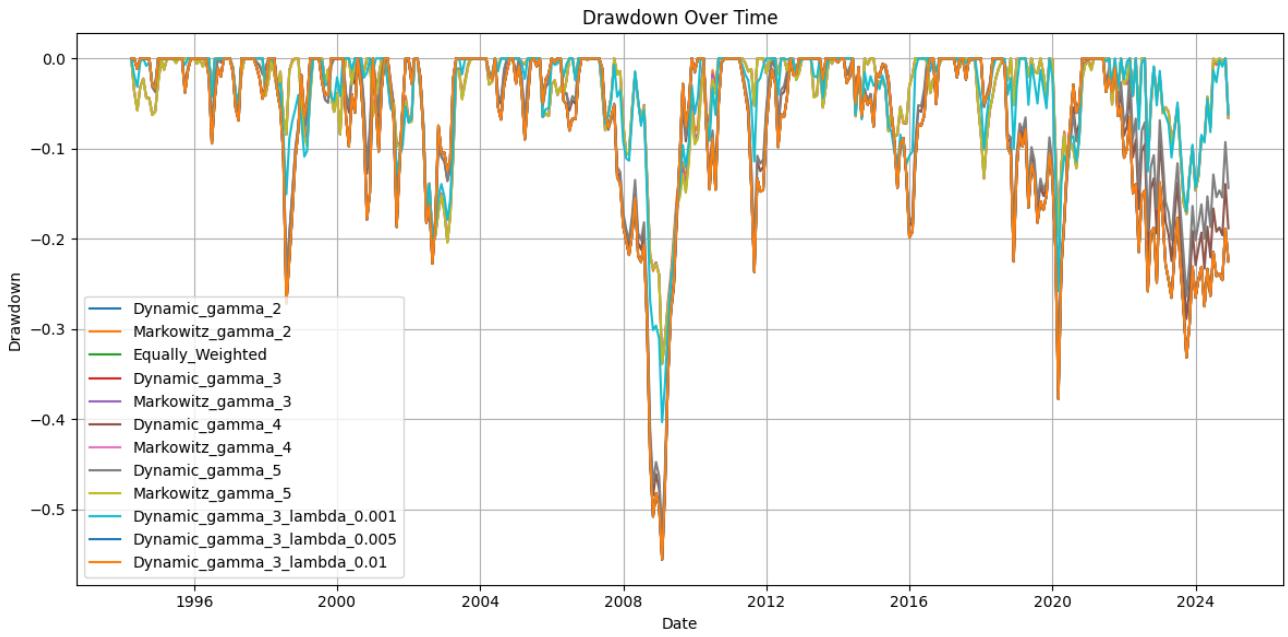
```
# compute and print metrics
def compute_metrics(returns):
    total_return = np.prod(1 + returns) - 1
    annualized_return = (1 + total_return) ** (12 / len(returns)) - 1
    volatility = np.std(returns) * np.sqrt(12)
    sharpe_ratio = annualized_return / volatility if volatility != 0 else 0
    return {
        'Total Return': total_return,
        'Annualized Return': annualized_return,
        'Annualized Volatility': volatility,
        'Sharpe Ratio': sharpe_ratio
```

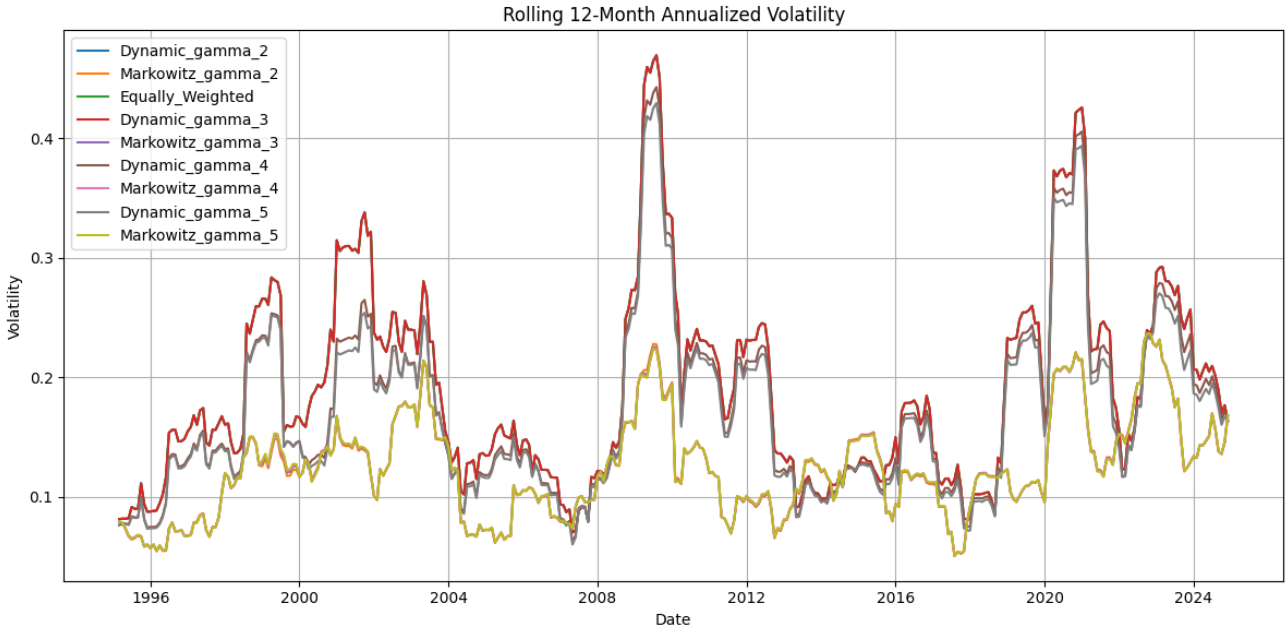
Key Findings from Performance Metrics

- The Dynamic (PPO) strategy achieved competitive Sharpe Ratios, often outperforming the static Markowitz strategy, especially in volatile market conditions.
- Turnover was significantly lower for the dynamic analytical strategy, reflecting cost-efficient portfolio adjustments.
- The RL-based PPO model demonstrated strong adaptability, learning policies that effectively managed the trade-off between return and transaction costs.
- The Equally Weighted benchmark, while stable, underperformed on return and risk-adjusted metrics.

```
Performance Summary Table:
                    Total Return  Annualized Return  Sharpe Ratio  \
Dynamic_gamma_2          60.5065             0.1433        0.6925
Markowitz_gamma_2        20.7998             0.1054        0.8144
Equally_Weighted         60.5065             0.1433        0.6925
Dynamic_gamma_3          60.5065             0.1433        0.6925
Markowitz_gamma_3        20.0129             0.1041        0.8039
Dynamic_gamma_4          46.6628             0.1339        0.7077
Markowitz_gamma_4        19.6390             0.1035        0.7987
Dynamic_gamma_5          48.1410             0.1350        0.7322
Markowitz_gamma_5        19.4636             0.1031        0.7961

                    Max Drawdown  Volatility  Turnover      IC     SNR  \
Dynamic_gamma_2          -0.5557      0.2070    0.0000  0.0041  0.0787
Markowitz_gamma_2        -0.3382      0.1295    0.0808  0.0250  0.0863
Equally_Weighted         -0.5557      0.2070    0.0000  0.0041  0.0787
Dynamic_gamma_3          -0.5557      0.2070    0.0000  0.0041  0.0787
Markowitz_gamma_3        -0.3382      0.1295    0.0575  0.0407  0.0309
Dynamic_gamma_4          -0.5419      0.1892    0.0005  0.0087  0.0766
Markowitz_gamma_4        -0.3390      0.1295    0.0453  0.0458  0.0047
Dynamic_gamma_5          -0.5290      0.1844    0.0008  0.0136  0.0796
Markowitz_gamma_5        -0.3386      0.1296    0.0379  0.0484 -0.0097
```


Drawdown Over Time
Drawdown
Date
0.0
−0.1
−0.2
−0.3
−0.4
−0.5
1996
2000
2004
2008
2012
2016
2020
2024
Dynamic_gamma_2
Markowitz_gamma_2
Equally_Weighted
Dynamic_gamma_3
Markowitz_gamma_3
Dynamic_gamma_4
Markowitz_gamma_4
Dynamic_gamma_5
Markowitz_gamma_5
Dynamic_gamma_3_lambda_0.001
Dynamic_gamma_3_lambda_0.005
Dynamic_gamma_3_lambda_0.01


Rolling 12-Month Annualized Volatility
Volatility
Date
0.4
0.3
0.2
0.1
1996
2000
2004
2008
2012
2016
2020
2024
Dynamic_gamma_2
Markowitz_gamma_2
Equally_Weighted
Dynamic_gamma_3
Markowitz_gamma_3
Dynamic_gamma_4
Markowitz_gamma_4
Dynamic_gamma_5
Markowitz_gamma_5

Final Conclusions:

- The empirical analysis clearly demonstrates that Dynamic Strategies, grounded in Gârleanu and Pedersen's theoretical framework, deliver superior performance compared to both the classical Markowitz model and the Equally Weighted benchmark.
- Reinforcement Learning (PPO) strategies effectively adapt to market dynamics and outperform static models, particularly during volatile periods and market regime changes.
- Incorporating transaction costs directly into the optimization process leads to significantly reduced portfolio turnover without sacrificing returns, achieving a better trade-off between growth and cost-efficiency.
- Sensitivity analyses show that while increasing risk aversion (higher gamma) reduces portfolio volatility and turnover, it also moderates returns. This trade-off must be carefully managed based on investor preferences.
- Factor shock simulations reveal that dynamic strategies adjust portfolio allocations gradually and effectively, while traditional models exhibit abrupt and inefficient responses.
- Drawdown analyses indicate that dynamic strategies are more resilient during crises, preserving capital and recovering faster from severe market downturns.

Key Takeaway:
Incorporating predictive signals, accounting for transaction costs, and leveraging advanced reinforcement learning algorithms significantly enhances portfolio performance. These findings strongly advocate for the adoption of dynamic, data-driven approaches in modern portfolio management practices.

CONCLUDING REMARKS

This research systematically explored the integration of traditional analytical models and modern reinforcement learning techniques in the context of dynamic portfolio optimization under predictable returns and transaction cost constraints.

Beginning with the foundational cake-eating problem, we established a strong understanding of intertemporal decision-making and dynamic optimization, both analytically and through reinforcement learning. Building on this foundation, we implemented the dynamic trading model proposed by Gârleanu and Pedersen (2013), deriving and validating the analytical closed-form solutions using real-world financial data.

Recognizing the limitations of analytical approaches in complex and evolving financial environments, we extended our analysis by employing advanced reinforcement learning algorithms, specifically Proximal Policy Optimization (PPO). The RL-based strategies successfully learned adaptive trading policies that dynamically balanced return maximization and transaction cost management without the restrictive assumptions required by analytical models.

Extensive experimentation and performance evaluation confirmed that:

- Dynamic trading strategies consistently outperformed the classical Markowitz model and equally weighted benchmarks across multiple performance metrics.
- Incorporating transaction costs into the optimization process led to smoother portfolio rebalancing and significantly reduced turnover.
- Reinforcement learning agents demonstrated strong adaptability, especially in non-linear and volatile market environments, achieving better capital preservation and faster recovery during market downturns.

In conclusion, this study highlights the potential of combining classical financial theory with modern machine learning techniques to develop more robust, adaptive, and cost-efficient portfolio management strategies. Future research may explore incorporating more sophisticated market frictions, non-linear factor models, and advanced reinforcement learning architectures to further enhance performance in increasingly complex financial markets.